

The MediaTek logo is an orange parallelogram with the word "MEDIATEK" in white, bold, sans-serif capital letters.

MT8676 Android GPU User Manual

Version: 1.0
Release date: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-08-12	Lingxiao Wang	Official release

Table of Contents

Version History 2

Table of Contents 3

List of Figures 3

List of Tables 3

1 GPU 4

1.1 Overview 4

1.1.1 What is a GPU 4

1.1.2 Why GPU is Needed 4

1.1.3 How to Use the GPU 4

1.2 Architecture/Process Overview 5

1.2.1 Android Graphics System Framework 5

1.2.2 Arm Mali-G615 Architecture and Feature Support 5

1.3 Frequently Asked Questions/Troubleshooting 7

1.3.1 GPU Rendering Analysis 7

1.3.2 GPU Performance Analysis 9

Exhibit 1 Terms and Conditions 12

List of Figures

Figure 1-1. Android graphics system framework 5

Figure 1-2. Arm Mali-G615 architecture 6

Figure 1-3. MGD 8

Figure 1-4. MGD configuration 8

Figure 1-5. Establish connection 8

Figure 1-6. Streamline example 10

List of Tables

Table 1-1. Arm Mali-G615 feature support 6

1 GPU

1.1 Overview

This section mainly introduces the basic knowledge of the MT8676 Graphics Processing Unit (GPU). MT8676 GPU uses the Arm Mali-G615, with a computing power of 1.8T FLOPS.

1.1.1 What is a GPU

A GPU, also known as a graphics processor, visual processor, or display chip, is a microprocessor specialized in performing image and graphics-related computations on personal computers, workstations, gaming consoles, and some mobile devices such as tablets and smartphones.

1.1.2 Why GPU is Needed

The GPU, as the “heart” of the hardware graphics card, holds a status equivalent to that of the CPU in a computer system. The GPU can also be used as an important criterion to distinguish between 2D and 3D hardware graphics cards. 2D hardware graphics cards primarily use the CPU to handle features and 3D images, which is referred to as “software acceleration.”

The GPU parallel programming model is entirely different from the CPU serial programming model, resulting in many excellent algorithms on the CPU not being directly mappable to the GPU. Additionally, the GPU architecture is akin to a shared memory multiprocessor structure, leading to significant differences between parallel programs designed for the GPU and serial programs for the CPU. The GPU mainly employs key technologies such as cubic environment mapping, hardware Transform and Lighting (T&L), vertex blending, bump mapping, texture compression, and a dual-texture four-pixel 256-bit rendering engine.

Due to the highly parallel nature of graphics rendering tasks, the GPU can effectively improve processing power and memory bandwidth simply by increasing the number of parallel processing units and memory control units.

The design purposes of the GPU and CPU are fundamentally different. The CPU is designed to handle general-purpose tasks and thus has complex control units, whereas the GPU is primarily used to handle computationally intensive but less logically complex tasks. The processing units in the GPU can be more extensively utilized as execution units. Therefore, compared to the CPU, the GPU has unparalleled advantages in application scenarios characterized by large amounts of repetitive data set computations and frequent memory access.

1.1.3 How to Use the GPU

There are two ways to use the GPU. One way is for the developed application to call the GPU device through a general graphics library interface. The other way is for the GPU itself to provide an API programming interface, allowing the application to directly call the GPU device through the provided API.

Using the GPU through a general graphics library involves utilizing existing graphics function libraries such as OpenGL or DirectX. This method controls the GPU's internal shaders to perform the required computations by writing in a shading language.

Currently, the industry-recognized graphics programming interfaces are mainly OpenGL and DirectX. OpenGL is the preferred environment for developing interactive and portable 2D and 3D graphics applications and is the most widely used standard for graphics applications. Any environment that adheres to the OpenGL standard will produce the same visual effects. Similar to OpenGL, DirectX (Direct Extension) is also a graphics API. To meet the needs of GPU applications, DirectX defines new versions in a timely manner based on the expansion and progress of new GPU product features, providing functionalities almost in sync with those offered by the GPU.

1.2 Architecture/Process Overview

1.2.1 Android Graphics System Framework

Please refer to [Figure 1-1](#) for the Android graphics system framework:

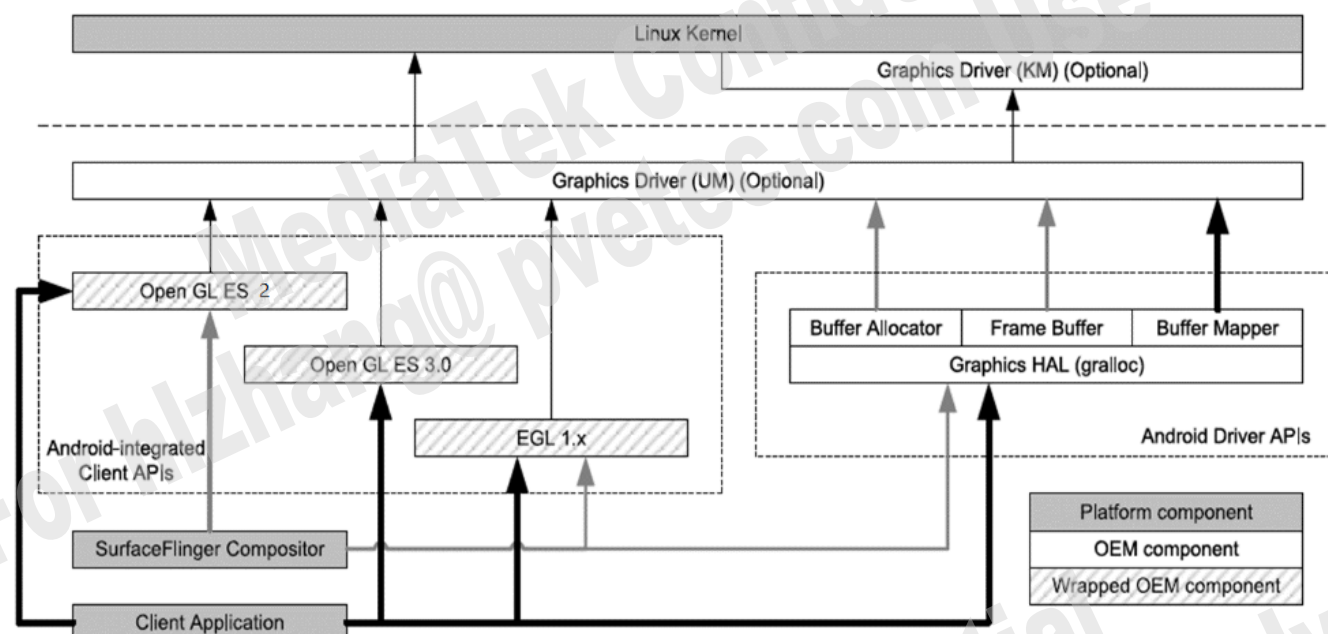


Figure 1-1. Android graphics system framework

1.2.2 Arm Mali-G615 Architecture and Feature Support

Please refer to [Figure 1-2](#) for the Arm Mali-G615 architecture.

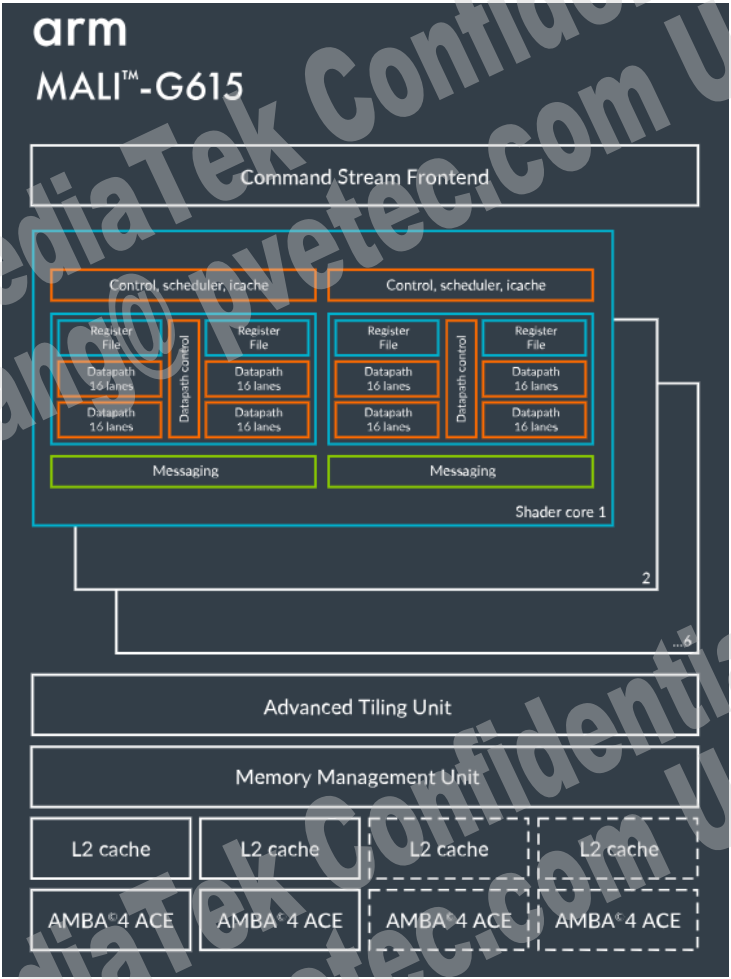


Figure 1-2. Arm Mali-G615 architecture

Please refer to Table 1-1 for the Arm Mali-G615 feature support.

Table 1-1. Arm Mali-G615 feature support

Features	Value	Description
Anti-Aliasing	4x MSAA 8x MSAA 16x MSAA	4x Multi-Sampling Anti-Aliasing (MSAA) with minimal performance drop.
API Support	- OpenGL[®] ES 1.1, 2.0, 3.1, 3.2 - Vulkan 1.1, 1.2, 1.3 - OpenCL[™] 1.1, 1.2, 2.0 Full Profile	Full support for next-generation and legacy 2D/3D graphics applications.
Adaptive Scalable Texture Compression (ASTC)	Low Dynamic Range (LDR) and High Dynamic Range (HDR). Supports both 2D and 3D images.	ASTC offers several advantages over existing texture compression schemes by improving image quality, reducing memory bandwidth and thus energy use.

Features	Value	Description
Arm Frame Buffer Compression (AFBC)	- Version 1.3.2 4x4 pixel block size	AFBC is a lossless image compression format that provides random access to pixel data to a 4x4 pixel block granularity. It is employed to reduce memory bandwidth both internally within the GPU and externally throughout the SoC.
Arm Fixed Rate Compression (AFRC)	- Version 1.0 4x4 pixel block size	AFRC is a lossy image compression format. AFRC can be used for compressing external texture inputs and framebuffer outputs from the GPU. Configurable compression ratio provides guaranteed bandwidth reduction for such surfaces and memory footprint saving.
Variable Rate Shading	- Pipeline, primitive and attachment shading rates - Up to 4x4 shading rate	Variable Rate Shading decouples fragment shading frequency from rasterization frequency, providing the opportunity to make energy savings while maintaining perceived visual quality.

1.3 Frequently Asked Questions/Troubleshooting

1.3.1 GPU Rendering Analysis

When screen rendering anomalies occur, the issue can generally be analyzed from three aspects: SF/HWC/Display, GPU, and APK. To determine if it is an SF/HWC/Display issue, first check the log for any display-related errors and proceed with further analysis based on the log. Additionally, the platform has two compositing methods, which can be done through OVL or GPU. You can disable hardware OVL and force the use of GPU for compositing to check for anomalies. Finally, you can use the screenrecord command to record the screen and check if the recording also shows rendering anomalies. If it is determined to be an SF/HWC/Display issue, you can contact the relevant owner for further analysis.

For GPU issues, search the log for errors related to keywords such as Mali/EGL/GLES/HWUI and proceed with further analysis based on the errors. You can also use some debugging tools, such as GPUD and Graphics API Debugger, Mali Graphics Debugger, RenderDoc, etc. These tools can help analyze the problem. Additionally, you can conduct comparative experiments related to the GPU.

For APK issues, you need to analyze together with the APK team to determine if there are problems with using GL interfaces during rendering or if incorrect textures are being passed for rendering.

1.3.1.1 MGD Usage Examples

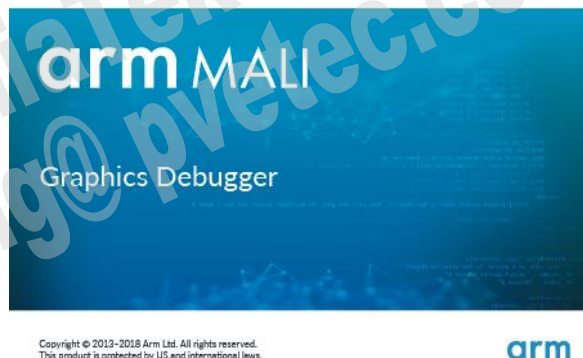


Figure 1-3. MGD

Mali Graphics Debugger (MGD) is a specialized debugging tool for Mali GPUs that can capture and analyze the rendering process of the target process's GPU. The following commands are required to prepare the platform:

```
adb root;
adb remount;
adb shell setenforce 0
```

Configure MGD according to the sequence highlighted in red in Figure 1-4. After configuration, you need to set the property to MGD using the following command:

```
adb shell "setprop vendor.debug.gpu.provider 'mgd' "
adb shell "getprop vendor.debug.gpu.provider"
adb shell "stop;start"
```

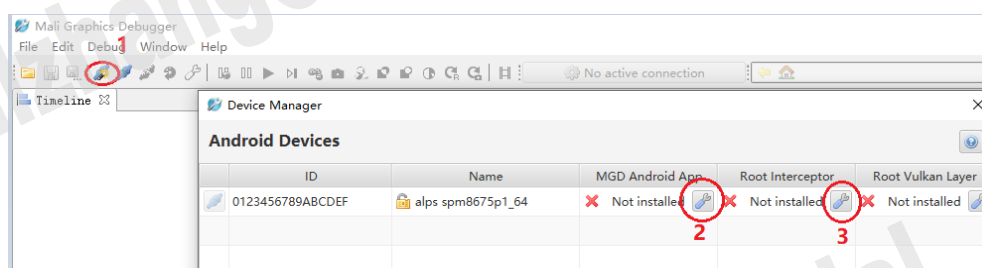


Figure 1-4. MGD configuration

Finally, establish the connection between the platform and MGD as shown in Figure 1-5. Once the connection is established, you can capture the rendering process of the target process.

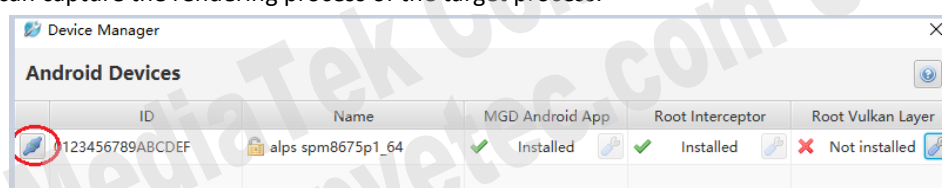


Figure 1-5. Establish connection

1.3.1.2 GPU-Related Comparative Experiments

Common comparative experiments are as follows:

- (1) Whether the issue can be reproduced on a Google reference device.
- (2) Whether the issue is related to the Android system version.
- (3) Whether the issue can be reproduced on the previous generation GPU framework (Midgard/Bifrost) platform.
- (4) Whether the issue is related to the GPU driver version.
- (5) Whether the issue can be reproduced with AFBC turned off.
- (6) Whether the issue can be reproduced by forcing glFinish.
- (7) Whether the issue can be reproduced with partial update turned off.
- (8) Whether the issue is related to ASTC or MSAA.
- (9) HWUI render pipeline switching experiment.
- (10) RenderEngine backend switching experiment.
- (11) Other aspects, etc.

1.3.2 GPU Performance Analysis

For analyzing GPU performance issues, there are generally three aspects to consider: GPU issues, APK issues, and issues related to other modules or the system. For GPU issues, you can check the Android log and kernel log for error logs with keywords such as Mali/EGL/GLES/HWUI and proceed with further analysis based on the logs. Tools like systrace or perfetto can be used to capture and analyze the problem scenario, and Arm Streamline can be used to check hardware execution to identify which specific part is affecting GPU performance. Targeted comparative experiments can also be conducted to break down the parts affecting performance.

For the APK part, systrace or perfetto can also be used to analyze whether the issue is caused by the APK. Issues related to other modules or the system can be analyzed through logs and flame graphs.

1.3.2.1 Common GPU Performance Comparison Experiments

Common comparative experiments for substandard GPU performance are as follows:

- (1) Whether the fixed performance mode passes.
- (2) Presence of unlimited frequency factors (e.g., thermal).
- (3) Whether it is related to the Power Policy strategy.
- (4) Whether it is related to driver overhead.
- (5) Whether it is related to memory bandwidth/GPU QoS.
- (6) Whether it is related to the GPU driver version.
- (7) Arm Mali Offline Compiler.
- (8) Other aspects, etc.:

1.3.2.2 Streamline Example

Streamline can be used to view the execution status and execution time of the GPU hardware.

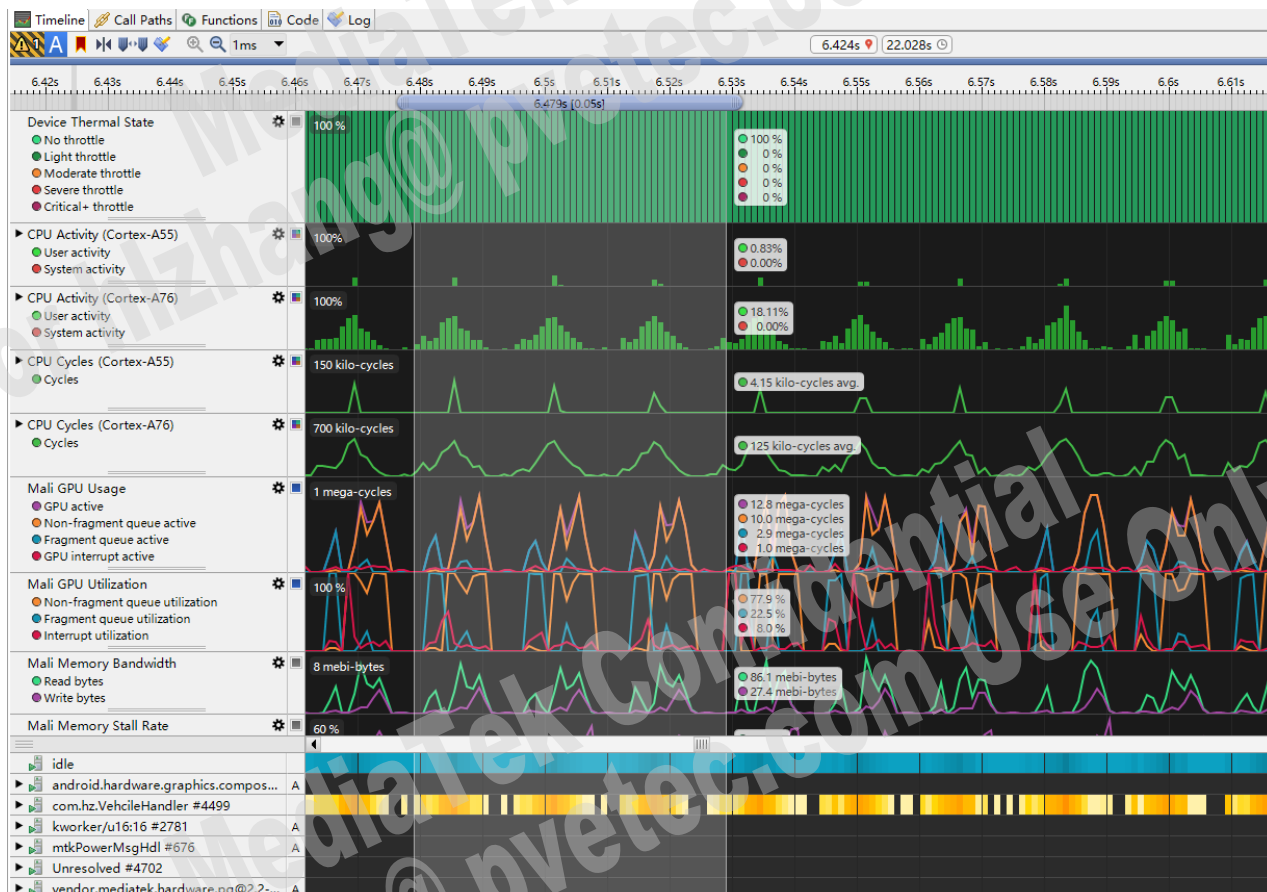


Figure 1-6. Streamline example

1.3.2.3 Performance Optimization Suggestions

The following are several suggestions for GPU performance optimization:

(1) Identify Performance Bottlenecks

Determine which specific part is affecting the platform's performance, such as the CPU, GPU (vertex or fragment), bandwidth, etc. Once the performance bottleneck is identified, targeted optimizations can be made.

(2) Heavy Vertex Load

A heavy vertex load can cause stuttering during GPU rendering. When using OpenGL, avoid having excessively large vertex attribute data.

(3) Heavy Fragment Load

Minimize overdraw and avoid using unnecessary precision and optimize mathematical expressions in shaders. Using simple and efficient statements in shaders can reduce the fragment load.

(4) Bandwidth Bottleneck

Optimization directions for bandwidth typically include: AFBC, ASTC, render size, mipmapping, pixel format, etc.

(5) Driver Overhead

Avoid using certain interfaces that cause the CPU and GPU to run serially (e.g., glReadPixels, glFinish). Optimize the number of gl interface calls per frame and advocate the use of VBO, EBO, VAO, etc.

(6) Developer Guide (From Arm Developer)

Arm GPU Best Practices Developer Guide

Link: <https://developer.arm.com/documentation/101897/0301?lang=en>

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.